

# cisco

## Introduction

Understanding the publicly exposed infrastructure of a target organization is the first step in any attack-surface mapping exercise. In this task, the objective is to identify the publicly routable IP addresses associated with **Cisco's homepage (cisco.com)** and explore the methodology behind expanding that initial footprint into actionable intelligence. This exercise mirrors the early reconnaissance phase of a real-world adversary operation, but conducted from a defensive, analytical perspective.

## Methodology

1. For this exercise we begin by scraping the cisco.com website and look for hyperlinks on the home page of the website.
  1. We will use the `wget --no-check-certificate https://www.cisco.com` command to download the webpage as a local `index.html` file for further enumeration.
  2. Then we try to identify patterns in the webpage for the hyperlinks in which we are interested.
  3. After downloading the homepage using `wget`, the next step is to **extract all hyperlinks** from the `index.html` file. Since the goal is to identify attack surface components and discover publicly routable Cisco-owned infrastructure, the approach focuses on isolating URLs, filtering out noise, and extracting only Cisco-related domains.

### 1. Viewing the downloaded HTML

```
ls
cat index.html
```

This confirms that the homepage was successfully saved and allows us to visually inspect the structure of the HTML for patterns. From this inspection, we observed that all hyperlinks begin with the `href=` attribute inside `<a>` tags.

```

root@kali:~# ls
cisco.sh Desktop domains.txt evilgrade Music Public Templates
cmds.txt Documents Downloads index.html Pictures sslstrip.log Videos
root@kali:~#
root@kali:~# cat index.html
<!DOCTYPE HTML>

<html lang="en-US" dir="ltr">
  <head>
    <meta charset="UTF-8"/>
    <meta name="HandheldFriendly" content="True"/>
    <meta name="MobileOptimized" content="320"/>
    <meta name="viewport" content="width=device-width, initial-scale=1.0"/>

    <title>AI Infrastructure, Secure Networking, and Software Solutions - Cisco</title>

```

## 2. Extracting lines that contain hyperlinks

```
cat index.html | grep "href="
```

This command returns every line in the HTML that contains an `href=` reference. Although messy, this provides the raw dataset we will refine in later steps.

```

root@kali:~#
root@kali:~# cat index.html | grep "href"
<link rel="canonical" href="https://www.cisco.com/">
<link rel="canonical" href="https://www.cisco.com/site/us/en/about/case-studies-customer-stories/the-adecco-group.html"/>
<link rel="alternate" hreflang="x-default" href="https://www.cisco.com/">
<link rel="alternate" hreflang="ja-jp" href="https://www.cisco.com/site/jp/ja/index.html"/>
<link rel="alternate" hreflang="uk-ua" href="https://www.cisco.com/site/ua/uk/index.html"/>
<link rel="alternate" hreflang="pt-br" href="https://www.cisco.com/site/br/pt/index.html"/>
<link rel="alternate" hreflang="zh-tw" href="https://www.cisco.com/site/tw/zh/index.html"/>
<link rel="alternate" hreflang="ru-ru" href="https://www.cisco.com/c/ru_ru/index.html"/>
<link rel="alternate" hreflang="cs-cz" href="https://www.cisco.com/site/cz/cs/index.html"/>

```

## 3. Extracting the domain portion of each URL

```
cat index.html | grep "href=" | cut -d"/" -f 3
```

Here:

- `cut -d"/" -f 3` splits each hyperlink using `/` as a delimiter
- Field 3 typically corresponds to the **domain name** in fully qualified URLs (e.g., `https://www.cisco.com/...` → `www.cisco.com`)

This step begins transforming unstructured HTML into meaningful OSINT artifacts.

```
root@kali:~#  
root@kali:~# cat index.html | grep "href" | cut -d"/" -f3  
www.cisco.com  
www.cisco.com  
www.cisco.com"  
www.cisco.com  
www.cisco.com  
www.cisco.com  
www.cisco.com  
www.cisco.com  
www.cisco.com
```

---

## 4. Filtering only Cisco-owned domains

Since the homepage contains many external references (fonts, images, CDNs, analytics, etc.), we need to keep only Cisco-controlled domains.

```
cat index.html | grep "href=" | cut -d"/" -f 3 | grep "cisco"
```

This filters the extracted domains so only entries containing the word *cisco* remain.

```
root@kali:~# cat index.html | grep "href" | cut -d"/" -f3 | grep "cisco" | tail  
newsroom.cisco.com  
blogs.cisco.com  
www.cisco.com  
blogs.cisco.com  
www.cisco.com  
www.ciscolive.com  
blogs.cisco.com  
blogs.cisco.com  
newsroom.cisco.com  
ciscocx.qualtrics.com
```

I also used the `tail` command just to show the last entries in the output because the entries at the top of the output were the same as above. However, this is not necessary for the task itself.

```
cat index.html | grep "href=" | cut -d"/" -f 3 | grep "cisco\.com"
```

Using `\.` escapes the dot to match `cisco.com` exactly, avoiding false positives.

```
www.cisco.com
www.cisco.com
www.cisco.com"
target.cisco.com"
smetrics.cisco.com"
target.cisco.com"
smetrics.cisco.com"
www.cisco.com" aria-labelledby="fw-c-header__logo-title">
blogs.cisco.com
blogs.cisco.com
software.cisco.com
www.cisco.com
community.cisco.com
www.cisco.com
www.cisco.com
newsroom.cisco.com
blogs.cisco.com
www.cisco.com
```

We still have many duplicates as well as some garbage entries which we need to clean up.

---

## 5. Cleaning leftover quotation marks

Some extracted domains may still include trailing `"`.

To clean these, we used:

```
cat index.html | grep "href=" | cut -d"/" -f 3 | grep "cisco\.com" | cut -d'"' -f1
```

This removes anything after the first `"`, leaving only the domain itself.

---

## 6. Sorting and extracting unique domains

```
cat index.html | grep "href=" | cut -d"/" -f 3 | grep "cisco\.com" | cut -d'"' -f1 | sort -u
```

This final command:

- sorts the list alphabetically
- removes duplicates with `-u`
- produces a clean, readable list of Cisco-owned domains referenced by the homepage

```
newsroom.cisco.com
root@kali:~# cat index.html | grep "href" | cut -d"/" -f3 | grep "cisco\.com" |
cut -d'"' -f1 | sort -u
blogs.cisco.com
community.cisco.com
newsroom.cisco.com
smetrics.cisco.com
software.cisco.com
target.cisco.com
www.cisco.com
```

---

## 7. Saving the results

```
cat index.html | grep "href=" | cut -d"/" -f 3 | grep "cisco\.com" | cut -d'"' -
f1 | sort -u > domains.txt
cat domains.txt
```

This stores all unique Cisco domains in a separate file, allowing further analysis such as:

- DNS lookups
- ASN (Autonomous System Number) enumeration
- IP address mapping
- Infrastructure pivoting

This completes the first stage of the attack surface discovery workflow.

---

## Extracting Publicly Routable IP Addresses from Discovered Domains

Once a clean list of Cisco-related domains was collected in `domains.txt`, the next stage of reconnaissance involved resolving each domain to its corresponding IPv4 address. This allows us to begin mapping the **public attack surface** of Cisco by identifying which IPs host or support their web infrastructure.

### 1 Automating DNS Resolution with a Bash Loop

To efficiently query every domain, we used a simple Bash script that iterates over each entry in `domains.txt` and performs a DNS lookup:

```
#!/bin/bash
for url in $(cat domains.txt); do
    host $url | grep "has address" | cut -d" " -f4
done
```

### 2 Explanation of the Script

- `for url in $(cat domains.txt); do`  
Loops through every domain extracted from the Cisco homepage.
- `host $url`  
Performs a DNS lookup using the `host` command, which returns all DNS records for that domain (A, AAAA, CNAME, MX, etc.).
- `grep "has address"`  
Filters for lines containing IPv4 A-records only, as these indicate **publicly routable IP addresses**.
- `cut -d" " -f4`  
Extracts the fourth field of the matching line, which contains the raw IPv4 address.

Example output of the `host` utility:

```
www.cisco.com has address 72.163.4.161
```

After processing, the script outputs only:

```
72.163.4.161
```

### 3 Purpose of This Step

This stage answers the core question of the exercise:

**What publicly routable IP addresses are associated with Cisco's homepage and its linked domains?**

By automating DNS lookups, we create a list of **live, reachable IPs** that can be further analyzed to understand:

- Which hosts serve Cisco's main website
- Whether Cisco uses CDNs (Content Delivery Networks)
- Whether any domains resolve to external infrastructure
- Which IPs belong to Cisco's autonomous system
- Whether the IPs are geographically distributed or load-balanced

These insights are the foundation for deeper attack-surface intelligence gathering.

### 4 Storing the Output

The script's output can be redirected into a file for further processing:

```
bash cisco.sh > ips.txt
```

This produces a clean list of IP addresses that can be used for:

- WHOIS / ASN enumeration
- Reverse DNS lookups
- CDN provider identification
- Port scanning (if allowed)
- Mapping infrastructure clusters

This concludes the IP extraction stage of the reconnaissance workflow.

---

## Addressing the Key Questions

### 1 How will you approach the challenge?

The challenge is approached using a **passive-to-light-active reconnaissance methodology**. The goal is to identify Cisco's publicly exposed web-facing infrastructure without performing intrusive actions.

The workflow consists of:

1. Downloading the homepage and extracting hyperlinks.
2. Parsing and filtering domains that belong to Cisco.
3. Resolving these domains to their corresponding public IP addresses.
4. Using these findings to build an initial understanding of Cisco's attack surface.

This mirrors early adversarial intelligence gathering, but executed entirely through OSINT and minimal DNS lookups.

---

### 4.2 What is the best starting point?

The best starting point is **DNS and HTML source-based enumeration**.

Cisco's homepage ( `cisco.com` ) acts as an entry point into the broader infrastructure; the references inside the page reveal:

- Subdomains
- External assets
- CDN Endpoints
- Internal service links
- Supporting platforms

Because the homepage already aggregates essential resources, extracting hyperlinks provides a natural and reliable starting set of domains that Cisco publicly exposes.

---

## 4.3 How are you pivoting to more intelligence?

Once initial domains and IPs are discovered, we can pivot into deeper intelligence using:

- **WHOIS lookups** → Identify owner organization, registrars, and contact info.
- **ASN enumeration** → Map the IPs to Cisco's allocated network ranges.
- **Reverse DNS lookups** → Identify hidden or undocumented hostnames.
- **Certificate transparency logs** → Discover additional subdomains or services.
- **CDN detection** → Identify if parts of the infrastructure are offloaded to Akamai, Cloudflare, etc.
- **Historical DNS records** → Understand legacy or deprecated infrastructure.

Each pivot reveals new assets, which can be recursively analyzed to construct a comprehensive attack surface map.

---

## 4.4 What are possible options for Cisco to limit your activities?

Cisco can limit reconnaissance or enumeration attempts by implementing:

- **Rate-limiting** on DNS, HTTP requests, and API endpoints.
- **CDN shielding** where the CDN absorbs traffic and hides real IPs.
- **Bot and crawler detection** using behavioral fingerprinting.
- **DNS response manipulation** (e.g., returning generic or internal addresses to known scanners).
- **WAF (Web Application Firewall) filtering** for non-browser signatures.
- **Sinkholing unused subdomains** so they do not reveal real infrastructure.
- **Centralizing assets behind load balancers** to reduce visible IP surface area.

These measures reduce the visibility and accessibility of Cisco's real infrastructure and complicate reconnaissance.

---

## 4.5 Is it possible for Cisco to detect you as an adversary at this stage?

It depends on the techniques used.

### **Passive activities (hard to detect):**

- Downloading HTML using `wget`.
- Parsing the file locally.

- OSINT on public DNS and WHOIS records.

None of these actions generate logs on Cisco's systems. These are effectively invisible.

### **Light active activities (detectable with effort):**

- DNS queries for many Cisco subdomains in a short period.
- Repeated A-record lookups from the same IP address.
- Unusual patterns in resolver logs (e.g., enumerating sequential subdomains).

Cisco could detect these behaviors **indirectly** through:

- Their authoritative DNS logs.
- Suspicious resolver traffic flagged by upstream providers.
- Known OSINT tool signatures (if not masked).

However, at this reconnaissance stage, detection is still **possible but unlikely**, especially if the enumeration is slow, distributed, or performed via third-party resolvers such as Google DNS or Cloudflare DNS.